

DATAFRAMES EM PYTHON – DATA CLEANING

É muito comum abrir datasets a partir de arquivos do tipo csv. O Exemplo abaixo faz uso dessa funcionalidade.

```
# Leitura do dataset a partir de um arquivo csv
import pandas as pd
import io
from google.colab import files

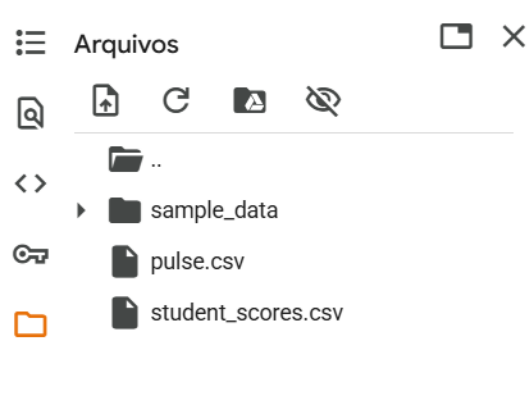
uploaded = files.upload()
df = pd.read_csv(io.BytesIO(uploaded['student_scores.csv']))
df.head()
```

Exercícios

```
# Qual a quantidade de estudo (em horas) média dos estudantes
# Qual o score médio dos estudantes?
# Retornar o número total de estudantes.
# Retornar o número total de estudantes com score abaixo da nota de aprovação
# Retornar o número total de estudantes com score acima da nota de aprovação
```

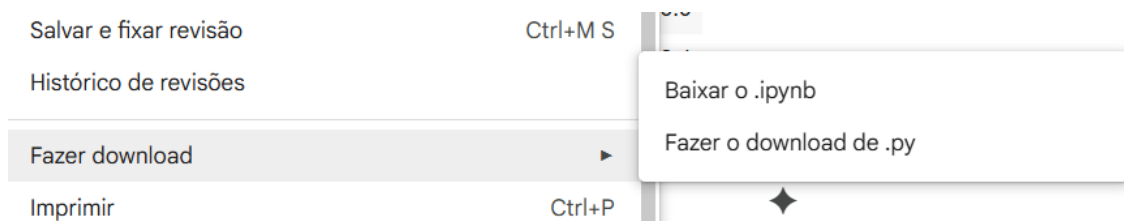
ARQUIVOS NO GOOGLE COLAB

Os arquivos ficam disponíveis no google colab após upload e podem ser manipulados pelo usuário.



NOTEBOOKS NO GOOGLE COLAB

Os notebooks podem ser baixados e importados para o google colab.



LIMPEZA DE DADOS

Cerca de 30 a 40% do tempo de um projeto de análise de dados é gasto obtendo e preparando os dados, e a limpeza de dados é parte fundamental nessa análise.

Os dados do dataset abaixo estão disponíveis no arquivo **pulse.csv**.

Duration		Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	NaN
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	'2020/12/26'	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
28	60	'2020/12/28'	103	132	NaN
29	60	'2020/12/29'	100	132	280.0
30	60	'2020/12/30'	102	129	380.3
31	60	'2020/12/31'	92	115	243.0

```
# Cleaning data - abertura do dataset pulse
uploaded = files.upload()
pulse = pd.read_csv(io.BytesIO(uploaded['pulse.csv']))
pulse.head()
```

Ao imprimir o conteúdo do dataset, pode-se observar a existência de dados nulos e informações duplicadas.

```
# Imprime o dataset completo
print(pulse.to_string())
```

Limpeza dos dados nulos em um novo dataset.

```
# Apagando registros com dados nulos
# dropna() retorna um novo dataset sem a presença de nulos
new_pulse = pulse.dropna()

print(new_pulse.to_string())
```

Limpa os dados no mesmo dataset.

```
# inplace=True faz alterações no dataframe original
pulse.dropna(inplace = True)
```

Outros comandos de limpeza do dataset.

```
# preenche os dados nulos com o valor 130
pulse.fillna(130, inplace = True)
```

```
# preenche com valor apenas uma coluna específica
pulse.fillna({"Calories": 130}, inplace=True)
```

```
# preenche com a média uma coluna específica
x = pulse["Calories"].mean()
pulse.fillna({"Calories": x}, inplace=True)
```

```
# Altera para duracao 45 o registro cujo índice é 7
pulse.loc[7, 'Duration'] = 45
```

```
# Altera para 120 todos os itens com Duração > 120
for x in df.index:
    if pulse.loc[x, "Duration"] > 120:
        pulse.loc[x, "Duration"] = 120
```

```
# Apaga todos os itens com Duração > 120
for x in df.index:
    if pulse.loc[x, "Duration"] > 120:
        pulse.drop(x, inplace = True)
```

```
# Detecta duplicados
print(pulse.duplicated())
```

```
# Remove duplicados
pulse.drop_duplicates(inplace = True)
```

Tente detectar dados duplicados novamente e confirme que os dados foram removidos.

EXERCÍCIOS

Mostre o **histograma** do dataset pulse e mostre **estatísticas** das colunas dos dados.

Creating DataFrames

	a	b	c
1	4	5	6
2	5	8	11
3	6	9	12

```
df = pd.DataFrame({
    "a": [4, 5, 6],
    "b": [7, 8, 9],
    "c": [10, 11, 12],
    index = [1, 2, 3]})
```

Specify values for each column.

```
df = pd.DataFrame([
    [4, 7, 10],
    [5, 8, 11],
    [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

	a	b	c
1	4	5	6
2	5	8	11
3	6	9	12

```
df = pd.DataFrame({
    "a": [4, 5, 6],
    "b": [7, 8, 9],
    "c": [10, 11, 12]},
    index = pd.MultiIndex.from_tuples(
        [('d', 1), ('d', 2),
         ('e', 2)], names=['n', 'v']))
```

Create DataFrame with a MultiIndex

Method Chaining

Most pandas methods return a DataFrame so that another pandas method can be applied to the result. This improves readability of code.

```
df = (pd.melt(df)
      .rename(columns={
          'variable': 'var',
          'value': 'val'})
      .query('val >= 200'))
```

Summarize Data

```
df['w'].value_counts()
Count number of rows with each unique value of variable
```

```
len(df)
# of rows in DataFrame.
```

```
df.shape
Tuple of # of rows, # of columns in DataFrame.
```

```
df['w'].nunique()
# of distinct values in a column.
```

```
df.describe()
Basic descriptive statistics for each column (or GroupBy).
```

pandas provides a large set of **summary functions** that operate on different kinds of pandas objects (DataFrame columns, Series, GroupBy, Expanding and Rolling (see below)) and produce single values for each of the groups. When applied to a DataFrame, the result is returned as a pandas Series for each column. Examples:

sum() Sum values of each object.	min() Minimum value in each object.
count() Count non-NA/null values of each object.	max() Maximum value in each object.
median() Median value of each object.	mean() Mean value of each object.
quantile([0.25, 0.75]) Quantiles of each object.	var() Variance of each object.
apply(function) Apply function to each object.	std() Standard deviation of each object.

Group Data

```
df.groupby(by="col")
Return a GroupBy object, grouped by values in column named "col".
```

```
df.groupby(level="ind")
Return a GroupBy object, grouped by values in index level named "ind".
```

All of the summary functions listed above can be applied to a group. Additional GroupBy functions:

```
size()
Size of each group.
```

```
agg(function)
Aggregate group using function.
```

Windows

```
df.expanding()
Return an Expanding object allowing summary functions to be applied cumulatively.
```

```
df.rolling(n)
Return a Rolling object allowing summary functions to be applied to windows of length n.
```

Tidy Data – A foundation for wrangling in pandas

In a tidy data set:

- Each variable is saved in its own column
- Each observation is saved in its own row

Tidy data complements pandas's **vectorized operations**. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.

Reshaping Data – Change layout, sorting, reindexing, renaming

```
pd.melt(df)
Gather columns into rows.
```

```
df.pivot(columns='var', values='val')
Spread rows into columns.
```

```
pd.concat([df1, df2])
Append rows of DataFrames
```

```
pd.concat([df1, df2], axis=1)
Append columns of DataFrames
```

```
df.sort_values('mpg')
Order rows by values of a column (low to high).
```

```
df.sort_values('mpg', ascending=False)
Order rows by values of a column (high to low).
```

```
df.rename(columns={'year': 'year'})
Rename the columns of a DataFrame
```

```
df.sort_index()
Sort the index of a DataFrame
```

```
df.reset_index()
Reset index of DataFrame to row numbers, moving index to columns.
```

```
df.drop(columns=['Length', 'Height'])
Drop columns from DataFrame
```

Subset Observations - rows

```
df[df.Length > 7]
Extract rows that meet logical criteria.
```

```
df.drop_duplicates()
Remove duplicate rows (only considers columns).
```

```
df.sample(frac=0.5)
Randomly select fraction of rows.
```

```
df.sample(n=10)
Randomly select n rows.
```

```
df.nlargest(n, 'value')
Select and order top n entries.
```

```
df.nsmallest(n, 'value')
Select and order bottom n entries.
```

```
df.head(n)
Select first n rows.
```

```
df.tail(n)
Select last n rows.
```

Subset Variables - columns

```
df[['width', 'length', 'species']]
Select multiple columns with specific names.
```

```
df['width'] or df.width
Select single column with specific name.
```

```
df.filter(regex='regex')
Select columns whose name matches regular expression regex.
```

Using query

query() allows Boolean expressions for filtering rows.

```
df.query('Length > 7')
df.query('Length > 7 and Width < 8')
df.query('Name.str.startswith("abc")', engine='python')
```

Subsets - rows and columns

Use **df.loc[]** and **df.iloc[]** to select only rows, only columns or both.

Use **df.at[]** and **df.iat[]** to access a single value by row and column.

First index selects rows, second index columns.

```
df.iloc[10:20]
Select rows 10-20.
```

```
df.iloc[:, [1, 2, 5]]
Select columns in positions 1, 2 and 5 (first column is 0).
```

```
df.loc[:, 'x2': 'x4']
Select all columns between x2 and x4 (inclusive).
```

```
df.loc[df['a'] > 10, ['a', 'c']]
Select rows meeting logical condition, and only the specific columns.
```

```
df.iat[1, 2]
Access single value by index
```

```
df.at[4, 'A']
Access single value by label
```

Logic in Python (and pandas)

<	Less than	!=	Not equal to
>	Greater than	df.column.isin(values)	Group membership
==	Equals	pd.isnull(obj)	is NaN
<=	Less than or equals	pd.notnull(obj)	is not NaN
>=	Greater than or equals	A, I, ~, -, df.any(), df.all()	Logical and, or, not, xor, any, all

regex (Regular Expressions) Examples

^.	Matches strings containing a period '.'
^Length\$	Matches strings ending with word 'Length'
^Sepal\$	Matches strings beginning with the word 'Sepal'
^[1-5]\$	Matches strings beginning with '1' and ending with 1,2,3,4,5
^(?!Species)\$	Matches strings except the string 'Species'

Handling Missing Data

```
df.dropna()
Drop rows with any column having NA/null data.
```

```
df.fillna(value)
Replace all NA/null data with value.
```

Make New Columns

```
df.assign(Area=lambda df: df.Length*df.Height)
Compute and append one or more new columns.
```

```
df['Volume'] = df.Length*df.Height*df.Depth
Add single column.
```

```
pd.qcut(df.col, n, labels=False)
Bin column into n buckets.
```

pandas provides a large set of **vector functions** that operate on all columns of a DataFrame or a single selected column (a pandas Series). These functions produce vectors of values for each of the columns, or a single Series for the individual Series. Examples:

max(axis=1) Element-wise max.	min(axis=1) Element-wise min.
clip(lower=-10, upper=10) Trim values at input thresholds.	abs() Absolute value.

Combine Data Sets

```
adf + bdf =
```

x1	x2	x3
A	1	T
B	2	F
C	3	NaN

Standard Joins

```
pd.merge(adf, bdf, how='left', on='x1')
Join matching rows from bdf to adf.
```

```
pd.merge(adf, bdf, how='right', on='x1')
Join matching rows from adf to bdf.
```

```
pd.merge(adf, bdf, how='inner', on='x1')
Join data. Retain only rows in both sets.
```

```
pd.merge(adf, bdf, how='outer', on='x1')
Join data. Retain all values, all rows.
```

Filtering Joins

```
adf[adf.x1.isin(bdf.x1)]
All rows in adf that have a match in bdf.
```

```
adf[~adf.x1.isin(bdf.x1)]
All rows in adf that do not have a match in bdf.
```

```
ydf + zdf =
```

x1	x2	x3
A	1	T
B	2	C
C	3	D

Set-like Operations

```
pd.merge(ydf, zdf)
Rows that appear in both ydf and zdf (Intersection).
```

```
pd.merge(ydf, zdf, how='outer')
Rows that appear in either or both ydf and zdf (Union).
```

```
pd.merge(ydf, zdf, how='outer', indicator=True)
.query('merge == "left_only"')
.drop(columns=['merge'])
Rows that appear in ydf but not zdf (Setdiff).
```

Plotting

```
df.plot.hist()
Histogram for each column
```

```
df.plot.scatter(x='w', y='h')
Scatter chart using pairs of points
```

